

I. C++ Analyze

```
g++ -save-temps -masm=intel -o main main.cpp
```

I-a) Main Stage

{sample} **.cpp** – Windows, Linux, macOS

{input} source file = source code + directives

1. Preprocessing^[1]

{sample} **.i** – Windows and **.ii** – Linux, macOS

{output} translation unit = preprocessed source file

{rule} process - including header files, macro expansion, and conditional compilation - preprocessor directives!

2. Compiling^[2]

{sample} **.s** – Windows, Linux, macOS

{output} assembly language source code

{rule}

1. Frontend

{output} IR - intermediate representation

{rule} verify syntax and semantics through lexical, syntax, and semantic analysis

2. Middleend

{output} optimized IR

{rule} perform optimizations that are independent of CPU architecture through reachability analysis, constant propagation, relocation of computation, etc.

3. Backend

{output} target-dependent assembly code

{rule} perform optimizations that are specific for the CPU architecture through register allocation, instruction scheduling to keep parallel execution units busy

3. Assembling^[3]

{sample} **COFF.obj** – Windows, **ELF.o** – Linux, and **Mach-O.o** – macOS

{output} object file = metadata + [machine code ∈ object code]

{rule}

1. object code is usually relocatable, and not usually directly executable

2. assembly language is translated to machine code and the same machine code can be packaged in different object file formats

3. metadata contains linking or debugging information, such as stack unwinding information

4. Linking^[4]

{sample} **PE/COFF.exe** – Windows, **ELF** – Linux, and **Mach-O** – macOS

{output} executable file

{rule}

1. symbol resolution (*static*) - associating each symbol reference with exactly one symbol definition from the symbol tables of its input relocatable object files
2. section relocation - associating a memory location with each symbol definition, and then modifying all of the references to those symbols so that they point to this memory location
3. **static libraries** contribute their code to the output and **shared libraries** generally contribute symbol or dependency information

5. Executing

{rule} dynamic loading - the Linux kernel finds the path to the program interpreter, usually the dynamic linker, then loads and runs the linker before passing control to main application!

I-b) Alternative Stage

4.1. Statically linked library^[5]

{sample} **.lib** – Windows and **.a** – Linux, macOS

{rule} collection of object files, usually comes in the form of `ar` archive files, augmented with a symbol table that accelerates the search for the needed file(s)

4.2. Dynamically linked library^[6]

{sample} **PE.dll** – Windows, **ELF.so** – Linux's shared object, and **Mach-O.dylib** – macOS

{rule} dynamic libraries and executables usually have the same format, so only one loader is needed, and an executable can be used as a shared library

[1]: [Translation unit](#)

[2]: [Compiler](#)

[3]: [Object file](#)

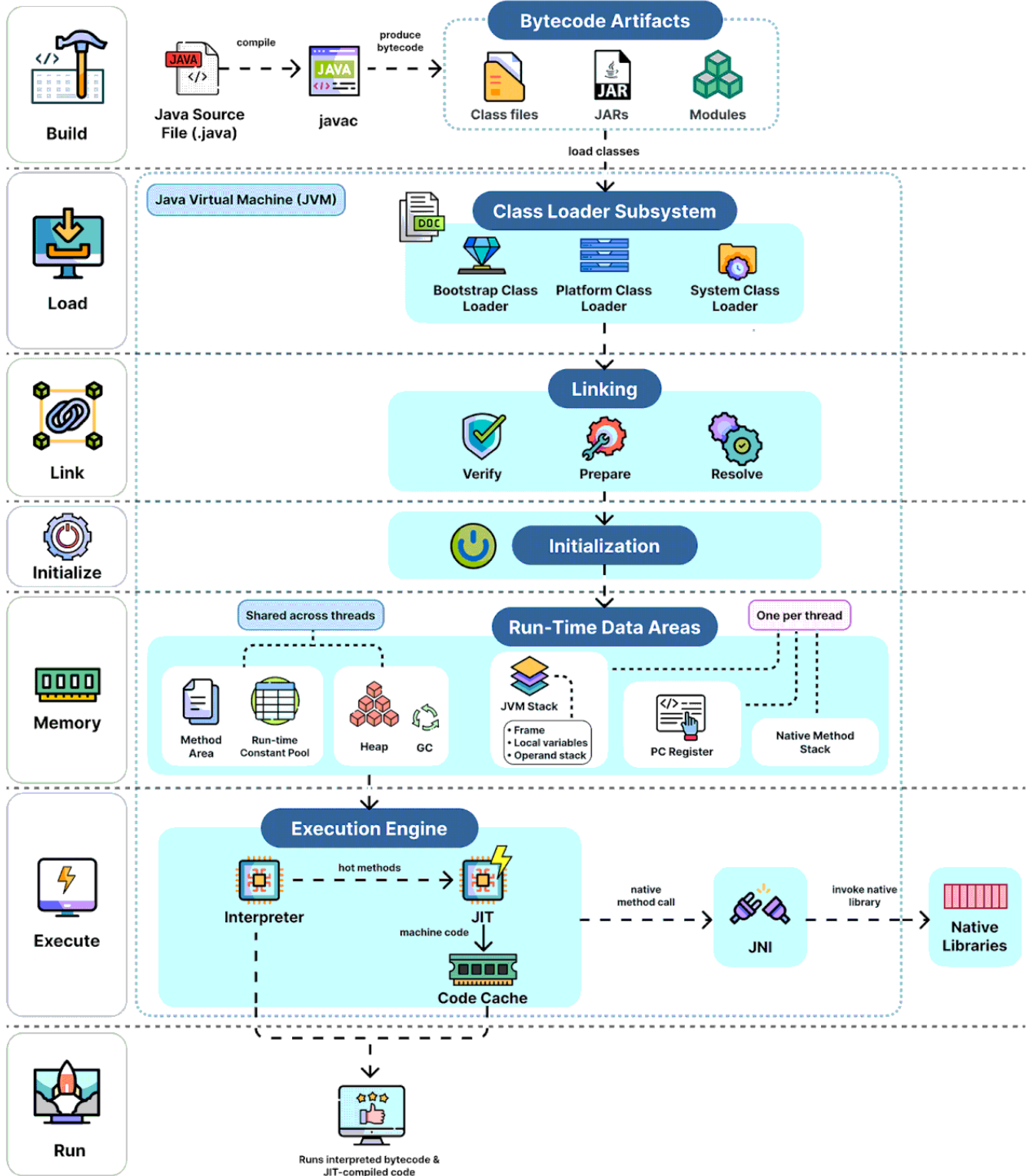
[4]: [Linking](#)

[5]: [Static library](#)

[6]: [Dynamic library](#)

II. Java Analyze^[7]

How the JVM Works



II-a) Main Stage

{input} .java source file

1. Compiling

{sample} .class files

{output} platform-independent [bytecode $\in$ object code]

2. Executing

{rule}

1. Class loading

{rule} load classes as needed, using parent delegation

2. Bytecode verification or linking

{rule} prepare allocates static fields with default values, and resolve symbolic references to their corresponding runtime entities

3. Initializing

{rule} assign static fields their initialized values and execute static initialization blocks

4. Garbage collection

{rule} reclaim heap memory occupied by objects that are no longer reachable

5. Runtime services and JIT compilation

{rule} the interpreter executes JVM bytecode, while the JIT compiler identifies frequently executed ("hot") code and may compile it into native machine code, which can be stored in the code cache. Native calls go through JNI to reach C/C++ libraries

II-b) Alternative Stage

2.1. Packaging

{sample} .jar archives and modules

[7]: [How the JVM Works](#)